

Algoritmos

Maia Ortiz

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy parecido a matlab y trabaja especialmente con variables que en general pueden ser matrices.

```
A=38
A=40
B=60
C=B-A
C
```

```
## [1] 20
```

```
##Uso de data sets o uso de bases de datos internos de R.
```

Si escribo en console data() se abre una serie de datos ya cargados en R que estan dados en tablas de las cuales podemos elegir columnas específicas si escribimos \$ despues del comando de información específico.

```
summary(cars)
```

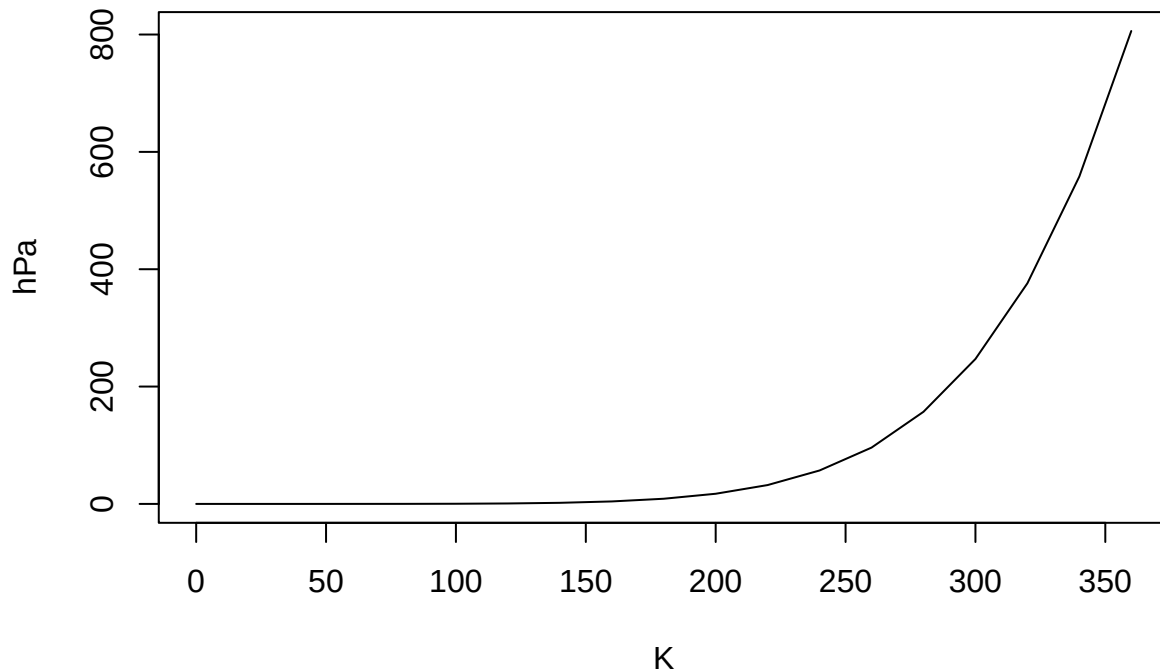
```
##      speed      dist
##  Min.   : 4.0    Min.   : 2.00
## 1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##   Mean  :15.4    Mean   : 42.98
## 3rd Qu.:19.0    3rd Qu.: 56.00
##   Max.  :25.0    Max.    :120.00
```

Including Plots

You can also embed plots, for example:

```
plot(pressure,type="l" , main="presión del gas ideal", ylab="hPa",xlab="K")
```

presión del gas ideal



##Comando Plot Comando que permite graficar cualquier fuente de datos que le asignemos, teniendo tambien la posibiidad de asignar nombres a las variables o hacer cambios al gráfico. En caso de no saber para que sirve un comando podemos escribir ?plot en la consola y se abrirá una pestaña que nos explicará.

##Funciones estadísticas

```
z1<-rnorm(350,21,5)
```

```
w1<-length(z1)
```

```
w1
```

```
## [1] 350
```

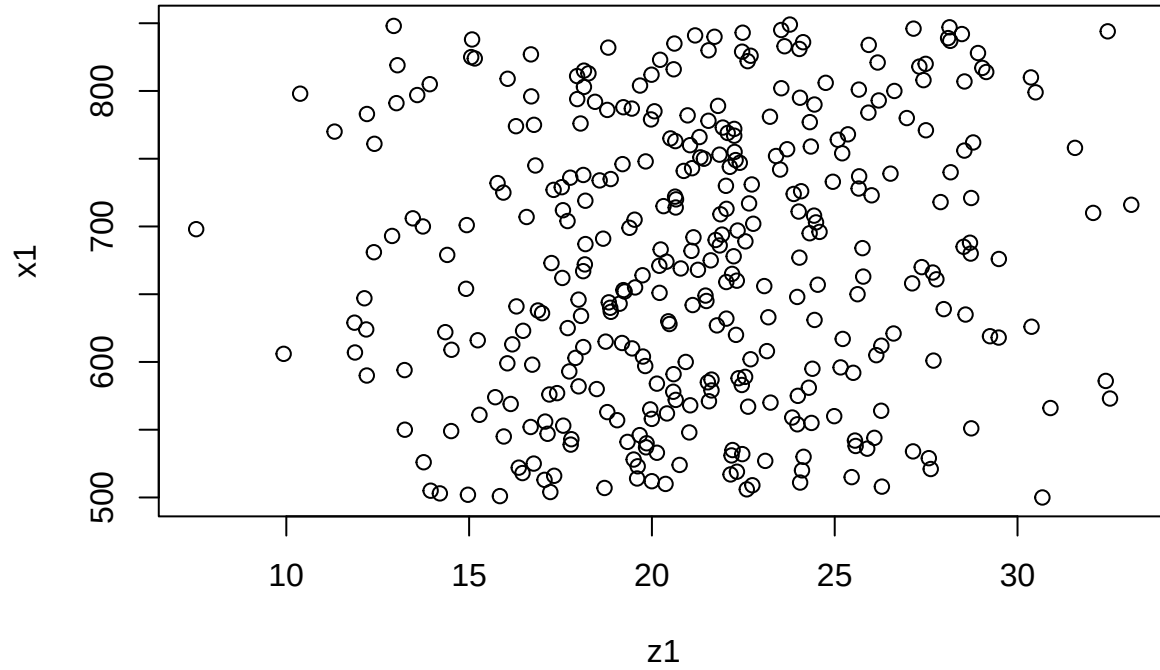
```
x1<-(500:849)
```

```
x1
```

```
## [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
## [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
## [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
## [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
## [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
## [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
```

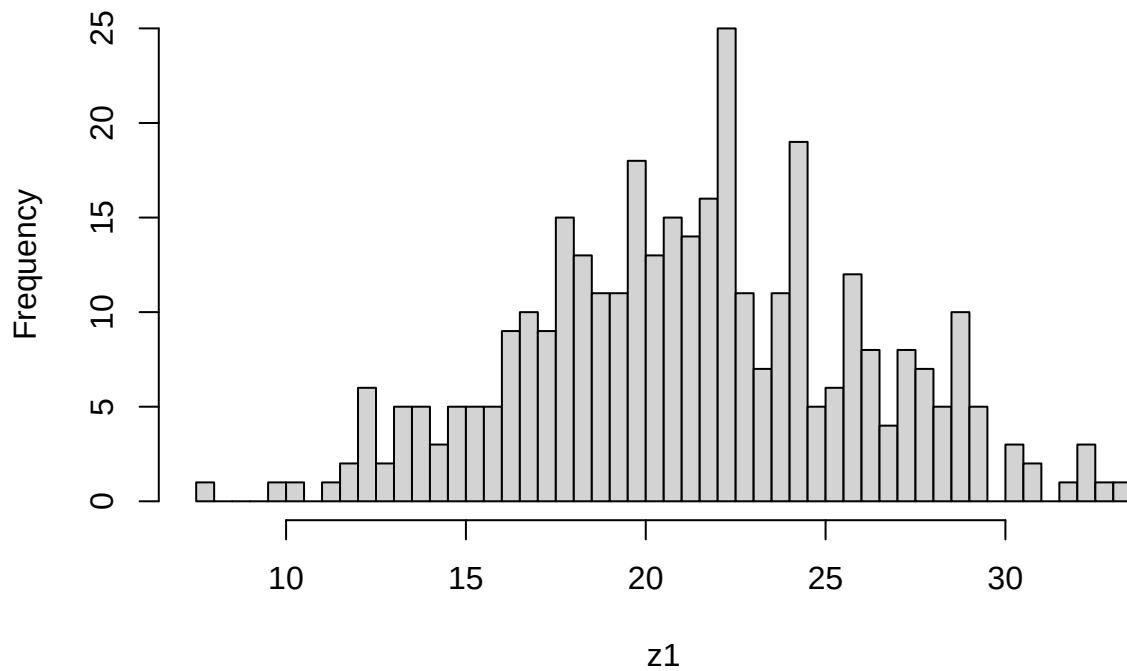
```
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```
plot(z1,x1) #comentario
```



```
hist(z1,main="Histograma de edades",breaks=50)
```

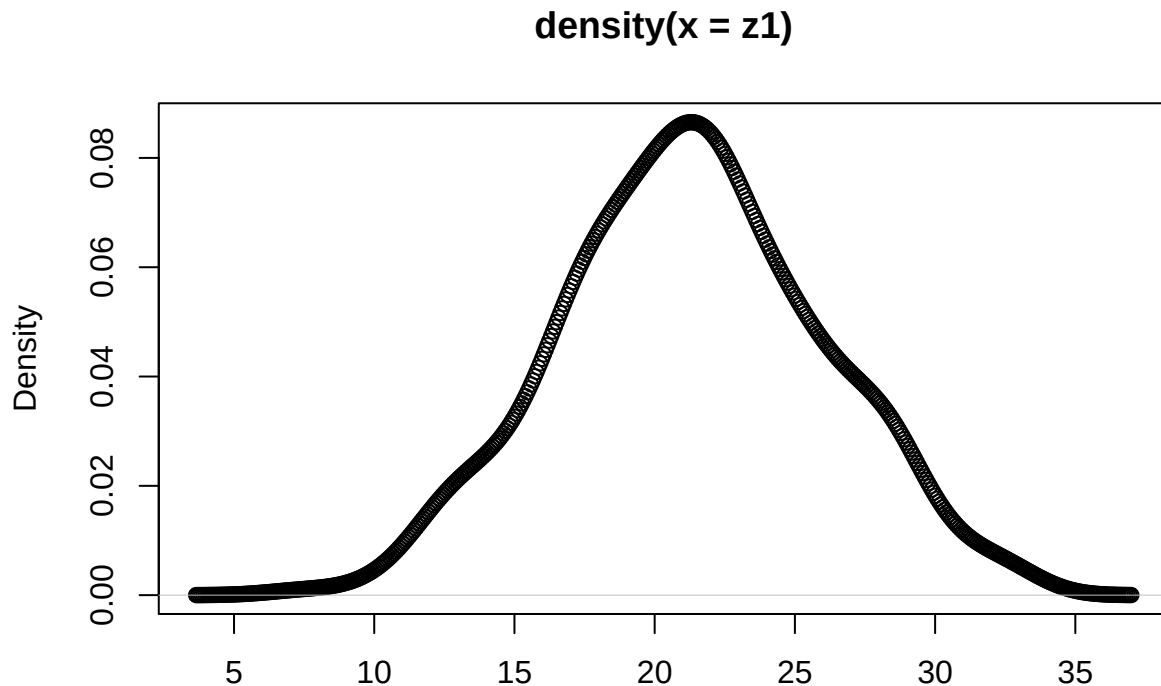
Histograma de edades



```
density(z1,type="d")
```

```
## Warning: In density.default(z1, type = "d") :  
## extra argument 'type' will be disregarded  
##  
## Call:  
## density.default(x = z1, type = "d")  
##  
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.295  
##  
##      x              y  
## Min.   : 3.651   Min.   :9.839e-06  
## 1st Qu.:11.987   1st Qu.:2.340e-03  
## Median :20.323   Median :2.115e-02  
## Mean   :20.323   Mean    :2.993e-02  
## 3rd Qu.:28.659   3rd Qu.:5.339e-02  
## Max.   :36.995   Max.    :8.654e-02
```

```
plot(density(z1),type="b")
```



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

#Ejercicio 1 de algoritmos

Consigna: Las últimas 3 cifras de mi DNI son 537 con estas crear una variable que tenga ese número

```
dni=537
```

Consigna: Crear un vector que tenga todos los números enteros desde el 1 hasta el 537.

```
secuenciadni<- (1:382)
secuenciadni
```

```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382
```

#Ejercicio 2: Calcular la suma de todos los valores del vector secuenciadni

```
total<-0
valorfinal<-length(secuenciadni)
for(i in 1:valorfinal){
  total<-total+i
total}
```

#Ejercicio 3: Repetir el ejercicio anterior pero en phyton

```
suma=0
for i in range (1,382):
  suma +=i

print(suma)
```

```
## 1
## 3
## 6
## 10
## 15
## 21
## 28
## 36
## 45
## 55
## 66
## 78
## 91
## 105
```

120
136
153
171
190
210
231
253
276
300
325
351
378
406
435
465
496
528
561
595
630
666
703
741
780
820
861
903
946
990
1035
1081
1128
1176
1225
1275
1326
1378
1431
1485
1540
1596
1653
1711
1770
1830
1891
1953
2016
2080
2145
2211
2278
2346

2415
2485
2556
2628
2701
2775
2850
2926
3003
3081
3160
3240
3321
3403
3486
3570
3655
3741
3828
3916
4005
4095
4186
4278
4371
4465
4560
4656
4753
4851
4950
5050
5151
5253
5356
5460
5565
5671
5778
5886
5995
6105
6216
6328
6441
6555
6670
6786
6903
7021
7140
7260
7381
7503

7626
7750
7875
8001
8128
8256
8385
8515
8646
8778
8911
9045
9180
9316
9453
9591
9730
9870
10011
10153
10296
10440
10585
10731
10878
11026
11175
11325
11476
11628
11781
11935
12090
12246
12403
12561
12720
12880
13041
13203
13366
13530
13695
13861
14028
14196
14365
14535
14706
14878
15051
15225
15400
15576

15753
15931
16110
16290
16471
16653
16836
17020
17205
17391
17578
17766
17955
18145
18336
18528
18721
18915
19110
19306
19503
19701
19900
20100
20301
20503
20706
20910
21115
21321
21528
21736
21945
22155
22366
22578
22791
23005
23220
23436
23653
23871
24090
24310
24531
24753
24976
25200
25425
25651
25878
26106
26335
26565

26796
27028
27261
27495
27730
27966
28203
28441
28680
28920
29161
29403
29646
29890
30135
30381
30628
30876
31125
31375
31626
31878
32131
32385
32640
32896
33153
33411
33670
33930
34191
34453
34716
34980
35245
35511
35778
36046
36315
36585
36856
37128
37401
37675
37950
38226
38503
38781
39060
39340
39621
39903
40186
40470

40755
41041
41328
41616
41905
42195
42486
42778
43071
43365
43660
43956
44253
44551
44850
45150
45451
45753
46056
46360
46665
46971
47278
47586
47895
48205
48516
48828
49141
49455
49770
50086
50403
50721
51040
51360
51681
52003
52326
52650
52975
53301
53628
53956
54285
54615
54946
55278
55611
55945
56280
56616
56953
57291

```
## 57630
## 57970
## 58311
## 58653
## 58996
## 59340
## 59685
## 60031
## 60378
## 60726
## 61075
## 61425
## 61776
## 62128
## 62481
## 62835
## 63190
## 63546
## 63903
## 64261
## 64620
## 64980
## 65341
## 65703
## 66066
## 66430
## 66795
## 67161
## 67528
## 67896
## 68265
## 68635
## 69006
## 69378
## 69751
## 70125
## 70500
## 70876
## 71253
## 71631
## 72010
## 72390
## 72771
```

#Ejercicio 4: ¿Cuánto tarda en correr el código del ejercicio 2?

```
inicio<-Sys.time()
total<-0
valorfinal<-length(secuenciadni)
for(i in 1:valorfinal){
  total<-total+i
total}

final<-Sys.time()
final-inicio
```

```
## Time difference of 0.004599333 secs
#Ejercicio 5: Repetir el ejercicio anterior utilizando la cabeza de Gauss
```

```
system.time({
  n <- length(secuenciadni)

  #Aplicamos la fórmula de Gauss
  total_gauss <- (n*(n+1))/2

  print(total_gauss)
})
```

```
## [1] 73153
```

```
##      user      system elapsed
##         0         0         0
```

PARTE DOS DE LOS EJERCICIOS

1.4 Generar un vector secuencia

```
# --- 1. Generar secuencia con bucle 'for' ---
# Primero inicializamos el vector
system.time({
  A <- 0
  for (i in 1:50000) {
    A[i] <- (i * 2)
  }
})
```

```
##      user      system elapsed
##  0.012    0.001    0.013
```

```
#Verificamos los resultados
head(A)
```

```
## [1]  2  4  6  8 10 12
```

```
tail(A)
```

```
## [1] 99990 99992 99994 99996 99998 100000
```

```
system.time({
  B <- seq(from = 2, to = 100000, by = 2)
})
```

```
##      user      system elapsed
##         0         0         0
```

```
# Verificamos los resultados
head(B)
```

```
## [1]  2  4  6  8 10 12
```

```
tail(B)
```

```
## [1] 99990 99992 99994 99996 99998 100000
```

1.5 Implementación de una serie Fibonacci o Fibonacci

```
# Definimos la cantidad de elementos de la serie que queremos calcular
n <- 30
```

```
# --- Método 1: Bucle 'for' con pre-asignación
system.time({
fib <- numeric(n)
fib[1] <- 0
fib[2] <- 1
for (i in 3:n) {
fib[i] <- fib[i-1] + fib[i-2]
}
})
```

```
##      user  system elapsed
##    0.003   0.000   0.003
```

```
# Ver los primeros resultados
print("Primeros números de la serie:")
```

```
## [1] "Primeros números de la serie:"
```

```
head(fib,10)
```

```
## [1] 0 1 1 2 3 5 8 13 21 34
```

```
# --- Método 2: Función Recursiva
fib_recursiva <- function(k) {
if (k == 0) return(0)
if (k == 1) return(1)
return(fib_recursiva(k - 1) + fib_recursiva(k - 2))
}
print("Tiempo ejecución recursiva:")
```

```
## [1] "Tiempo ejecución recursiva:"
```

```
system.time({
# Calculamos el elemento n de forma individual
resultado_rec <- fib_recursiva(n)
})
```

```
##      user  system elapsed
##    0.825   0.006   0.836
```

1.6 Definición matemática recurrente

```
# --- Resolver consigna con bucle 'while' ---
# Inicializamos la serie según la definición matemática
fib <- c(0, 1)
i <- 2 # Empezamos el conteo de iteraciones/posiciones
system.time({
# El bucle sigue mientras el último número generado sea <= 1.000.000
while (fib[i] <= 1000000) {
i <- i + 1
# Definición recurrente:  $f(n+1) = f(n) + f(n-1)$ 
fib[i] <- fib[i-1] + fib[i-2]
}
})
```

```
##      user  system elapsed
##    0.003   0.000   0.004
```

```
# --- Resultados ---
cat("El primer número mayor a 1.000.000 es:", fib[i], "\n")

## El primer número mayor a 1.000.000 es: 1346269
cat("Se necesitaron", i - 1, "iteraciones (pasos) para superarlo.\n")

## Se necesitaron 31 iteraciones (pasos) para superarlo.
# Mostramos los últimos elementos para verificar
tail(fib)

## [1] 121393 196418 317811 514229 832040 1346269
```

1.7 Ordenación de un vector por método burbuja

```
# --- 1. Creamos un vector desordenado ---
# Usamos un vector de 1000 números aleatorios
set.seed(123) # Para que el resultado sea siempre el mismo
vector_desordenado <- sample(1:1000, 1000)
# --- 2. Implementación del Método Burbuja ---
bubble_sort <- function(x) {
  n <- length(x)
  # El bucle externo recorre todo el vector
  for (i in 1:(n - 1)) {
    # El bucle interno compara pares adyacentes
    for (j in 1:(n - i)) {
      if (x[j] > x[j + 1]) {
        # Intercambio de posición (el "burbujeo")
        temp <- x[j]
        x[j] <- x[j + 1]
        x[j + 1] <- temp
      }
    }
  }
  return(x)
}
# --- 3. Medimos Performance del Burbuja vs R Nativo ---
cat("Tiempo con Método Burbuja:\n")
```

```
## Tiempo con Método Burbuja:
```

```
system.time({
  vector_ordenado_burbuja <- bubble_sort(vector_desordenado)
})
```

```
##      user  system elapsed
##    0.063   0.000   0.063
```

```
cat("\nTiempo con función sort() nativa de R:\n")
```

```
##
```

```
## Tiempo con función sort() nativa de R:
```

```
system.time({
  vector_ordenado_R <- sort(vector_desordenado)
})
```

```
##      user  system elapsed
```

```
##    0.000    0.000    0.001
```

1.8 La penitencia de Newton

```
# Definimos el límite de la suma (1*10^7 para que no tarde demasiado en R)
# Nota: 1*10^10 es un número extremadamente grande para un bucle for en R
n <- 1e7
# --- MÉTODO 1: La "Penitencia" (Bucle For / Fuerza Bruta) ---
# Este método suma uno por uno, como quería el profesor.
system.time({
  suma_iterativa <- 0
  for (i in 1:n) {
    suma_iterativa <- suma_iterativa + i
  }
})
```

```
##    user  system elapsed
##    0.166    0.000    0.168
```

```
cat("Resultado Suma Iterativa:", suma_iterativa, "\n")
```

```
## Resultado Suma Iterativa: 5e+13
```

```
# --- MÉTODO 2: La "Estrategia de Newton/Gauss" (Fórmula Analítica) ---
# Usamos la suma de una progresión aritmética: (n * (n + 1)) / 2
system.time({
  suma_analitica <- (n * (n + 1)) / 2
})
```

```
##    user  system elapsed
##      0        0        0
```

```
cat("Resultado Suma Analítica:", suma_analitica, "\n")
```

```
## Resultado Suma Analítica: 5e+13
```

Gráficos de violín

```
# 2.0 Comparación de Performance con Gráficos de Violín
# Instalamos y cargamos la librería necesaria
if(!require(microbenchmark)) install.packages("microbenchmark")
```

```
## Loading required package: microbenchmark
```

Loading required package: microbenchmark

```
if(!require(ggplot2)) install.packages("ggplot2")
```

```
## Loading required package: ggplot2
```

```
library(microbenchmark)
library(ggplot2)
# --- 2.1 Comparación: Burbuja vs Sort Nativo ---
set.seed(123)
vector_prueba <- sample(1:500, 500)
# Usamos microbenchmark para ejecutar los métodos varias veces y comparar
comparativa_sort <- microbenchmark(
  Burbuja = bubble_sort(vector_prueba),
  Nativo_R = sort(vector_prueba),
```



```

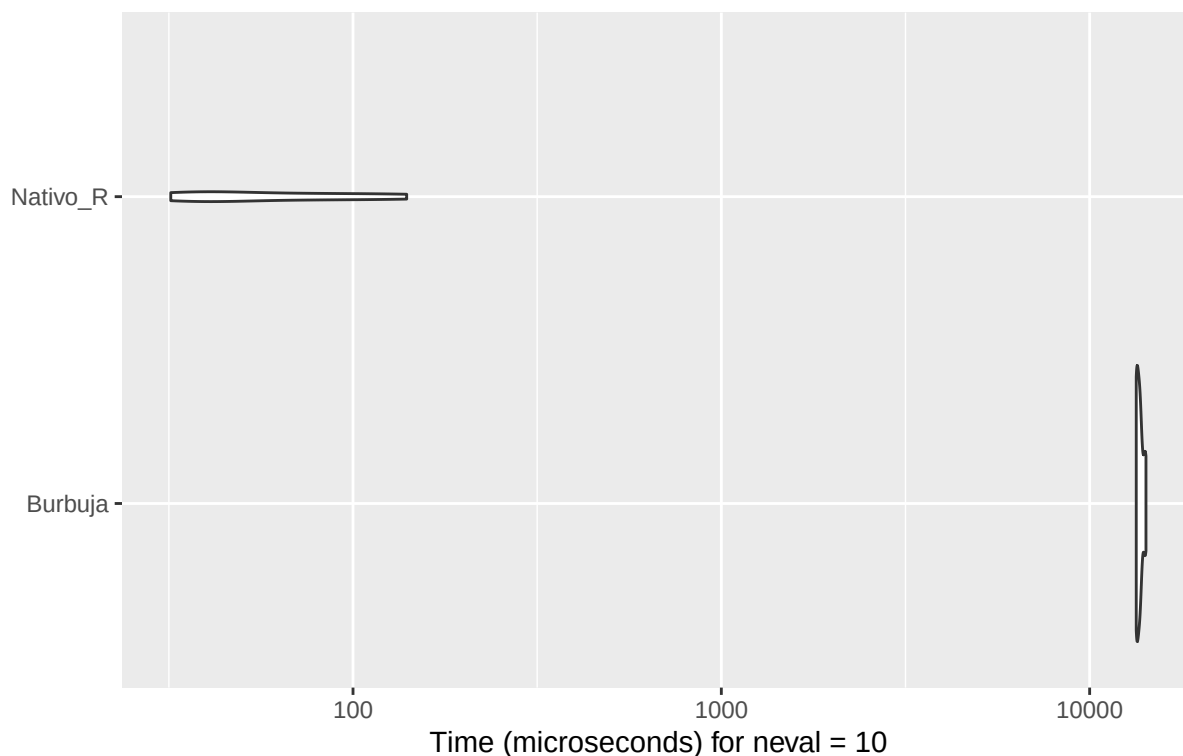
times = 10 # Lo corremos 10 veces para tener una muestra estadística
)
# Graficamos el comportamiento (Gráfico de Violín)
autoplot(comparativa_sort) +
labs(title = "Distribución de tiempos: Burbuja vs Sort Nativo",
      subtitle = "La escala suele ser logarítmica para apreciar la diferencia",
      y = "Tiempo de ejecución", x = "Algoritmo")

## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the microbenchmark package.
## Please report the issue at
## <https://github.com/joshuaulrich/microbenchmark/issues/>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

```

Distribución de tiempos: Burbuja vs Sort Nativo

La escala suele ser logarítmica para apreciar la diferencia



--- 2.2 K-means: Introducción Teórica y Medición ---

Introducción Teórica a K-means: El algoritmo de K-means es un método de agrupamiento (clustering) que particiona un conjunto de n observaciones en k grupos. Funciona de forma iterativa asignando cada punto al “centroide” más cercano y recalculando dichos centros hasta que las posiciones se estabilizan. Es ampliamente utilizado en minería de datos para encontrar patrones no visibles a simple vista.

```

# Medimos el rendimiento de kmeans con un dataset interno de R (iris)
comparativa_kmeans <- microbenchmark(
  Kmeans_3_centros = kmeans(iris[, 1:4], centers = 3),

```

```
Kmeans_5_centros = kmeans(iris[, 1:4], centers = 5),
times = 50
)
# Graficamos los resultados de K-means
autoplot(comparativa_kmeans) +
labs(title = "Performance del método K-means",
      subtitle = "Comparación variando la cantidad de clusters (k)",
      y = "Tiempo de ejecución", x = "Configuración")
```

Performance del método K-means

Comparación variando la cantidad de clusters (k)

